

Programmazione 1

Corso c

15/10/2023
> Le funzioni

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

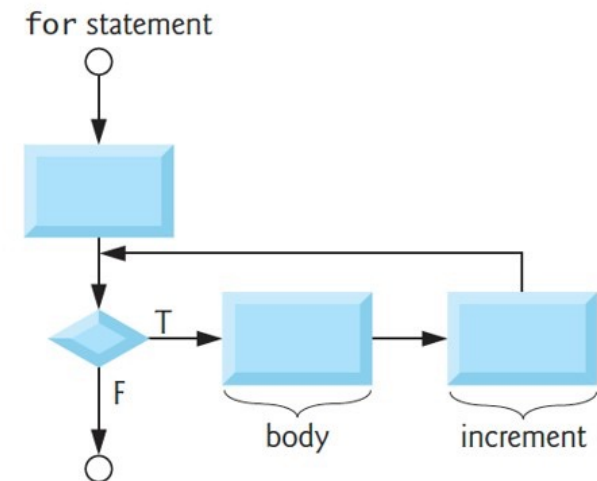
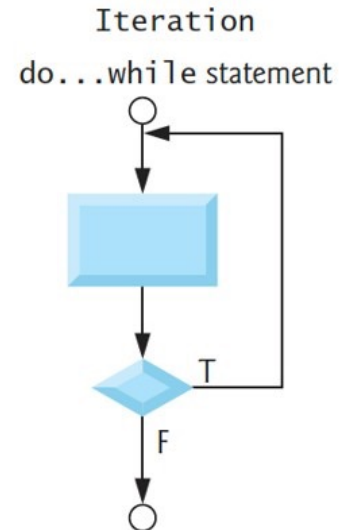
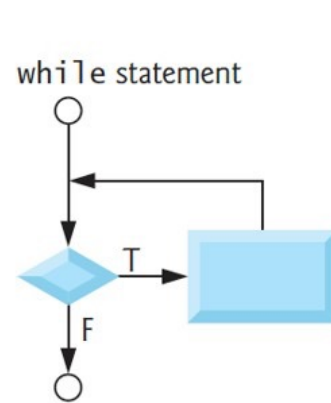
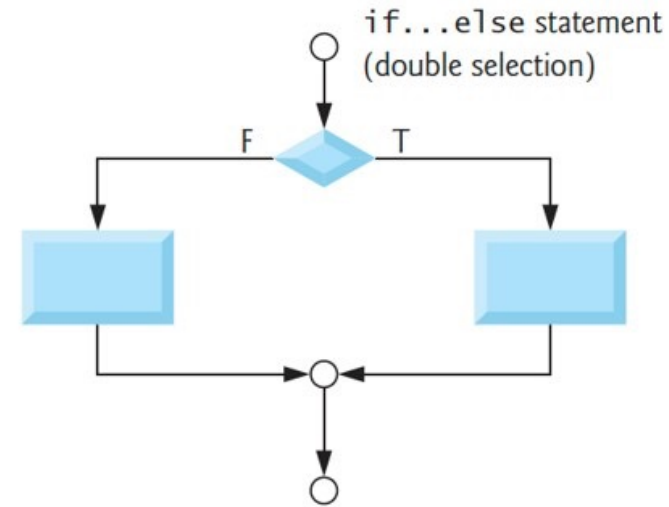
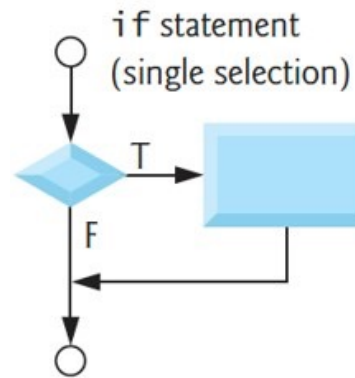
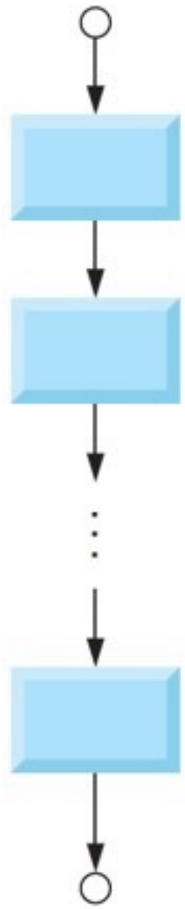
Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack

Strutture di controllo



Essenza del C

Qualsiasi forma di controllo necessaria in un programma C può essere espressa in termini di tre sole forme di controllo:

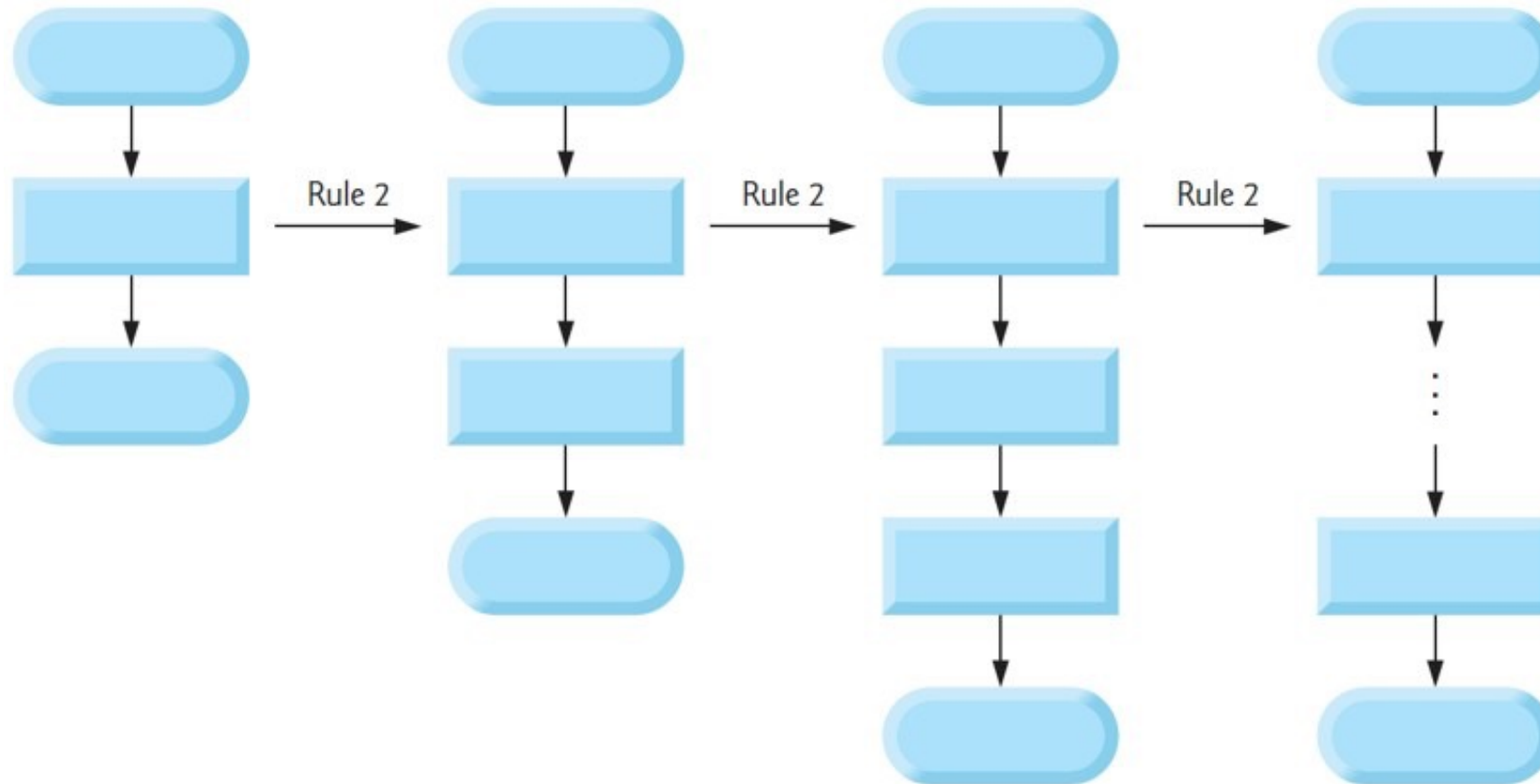
- sequenza
- istruzione if (selezione).
- istruzione while (iterazione).

Queste istruzioni di controllo possono essere combinate solo in due modi: **accatastamento** (stacking) e **annidamento** (nesting).

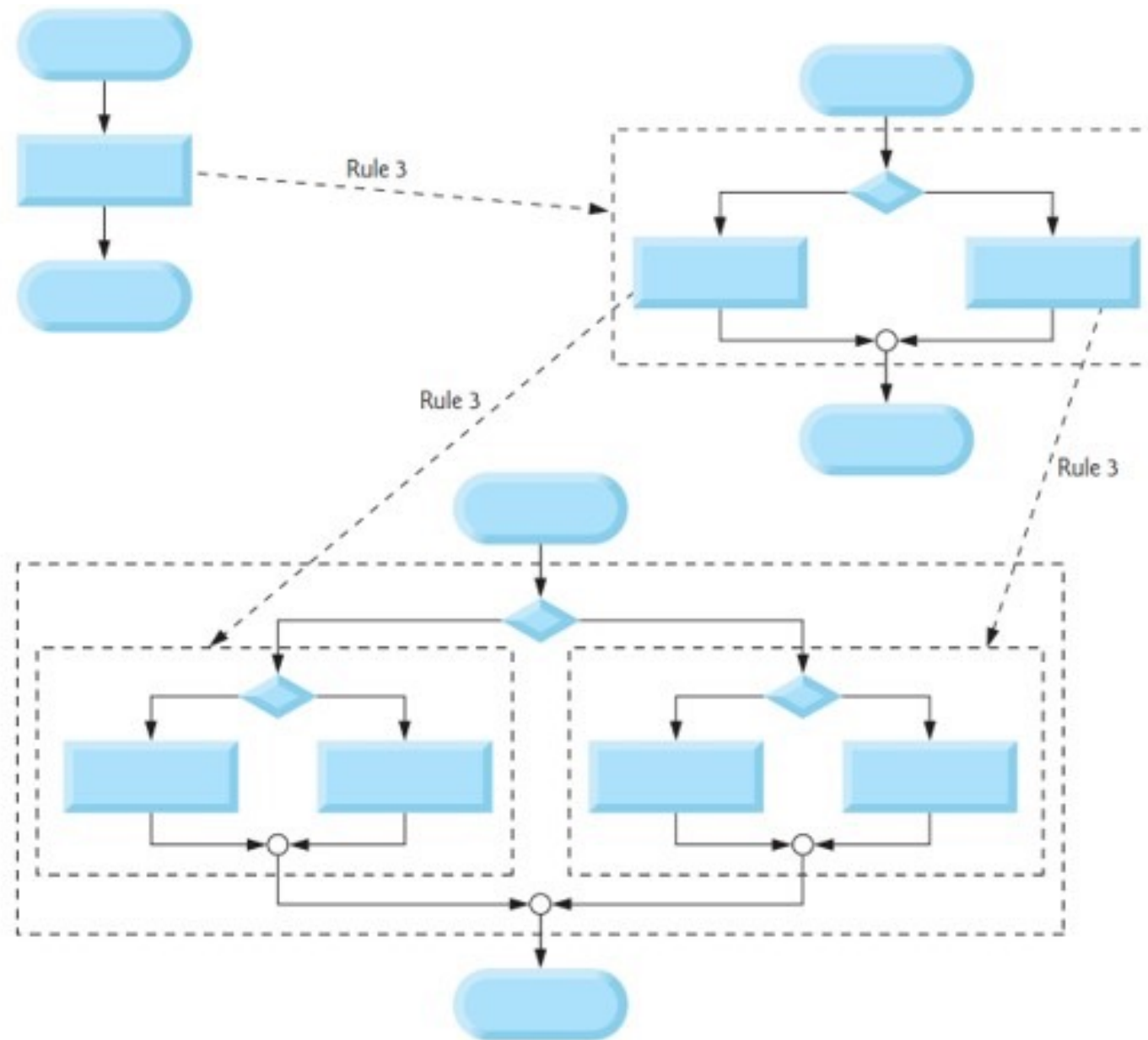
Regole per la creazione di programmi strutturati

1. Iniziate con il "diagramma di flusso più semplice"
2. **Regola dell'impilamento:** ogni rettangolo (azione) può essere sostituito da due rettangoli (azioni) in sequenza.
3. **Regola di annidamento:** ogni rettangolo (azione) può essere sostituito da qualsiasi istruzione di controllo.
4. Le regole 2 e 3 possono essere applicate tutte le volte che si vuole e in qualsiasi ordine.

Stacking



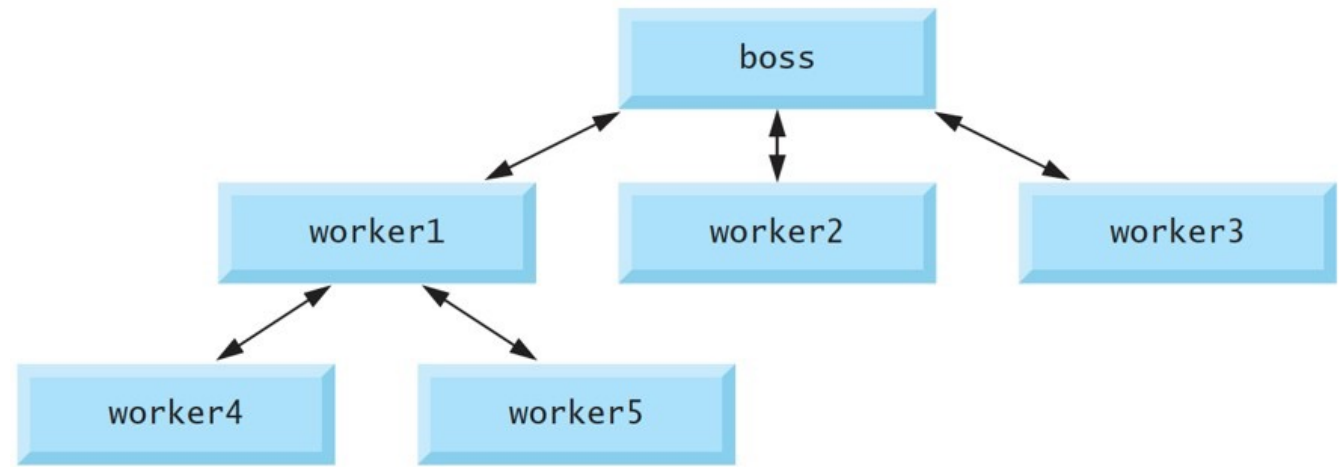
Nesting



Divide et impera

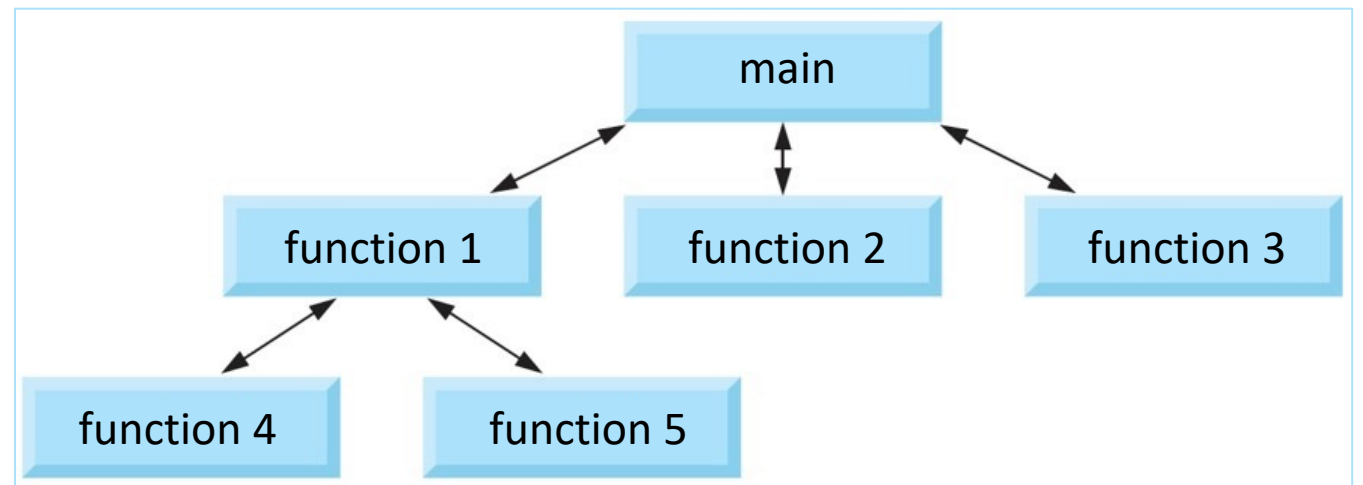
- L'idea della programmazione strutturata si fonde con l'idea della modularizzazione
- Si sviluppano e si mantengono programmi utilizzando pezzi più piccoli, ognuno dei quali è più gestibile del programma originale.
- Le funzioni seguono la stessa idea di modularizzare **blocchi di codice**, parametrizzando il loro funzionamento sulla base di valori di input
- *Software reusability*
- *Abstraction*

Management



Il capo chiede a un lavoratore di eseguire un compito e di riferire al termine dello stesso. Il capo non sa come il lavoratore esegue il compito designato. Il lavoratore può chiamare altri lavoratori e il capo non ne è a conoscenza.

Funzioni



La funzione **main** chiama (invoca) una **funzione** per eseguire un compito e per restituire (eventualmente) un valore al termine dello stesso. La **main** non sa come la funzione esegue il compito commissionato. La **funzione** può chiamare altre funzioni ma la **main** non ne è a conoscenza.

Funzione chiamante  Funzione chiamata

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack

Funzioni pre-definite: libreria standard

- Le funzioni della libreria matematica (header `math.h`) eseguono i calcoli matematici più comuni
- `printf("%.2f", sqrt(900.0)) ;`
 - Richiama la funzione `sqrt` per calcolare la radice quadrata di 900,0, quindi stampa il risultato come 30,00.
 - `sqrt` accetta un argomento `double` e restituisce un risultato `double`
 - Tutte le funzioni della libreria matematica che restituiscono valori in virgola mobile restituiscono il tipo `double`.
- Gli argomenti delle funzioni possono essere costanti, variabili o *espressioni*.
- La tabella seguente riassume alcune funzioni della libreria matematica del C:
 - Nella tabella, le variabili `x` e `y` sono di tipo `double`
 - Il C11 ha aggiunto le funzionalità per i numeri complessi tramite l'intestazione `complex.h`

math.h

Function	Description	Example
<code>sqrt (x)</code>	square root of x	<code>sqrt (900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt (x)</code>	cube root of x (C99 and C11 only)	<code>cbrt (27.0)</code> is 3.0
<code>exp (x)</code>	exponential function e^x	<code>exp (1.0)</code> is 2.718282 <code>exp (2.0)</code> is 7.389056
<code>log (x)</code>	natural logarithm of x (base e)	<code>log (2.718282)</code> is 1.0 <code>log (7.389056)</code> is 2.0
<code>log10 (x)</code>	logarithm of x (base 10)	<code>log10 (1.0)</code> is 0.0 <code>log10 (10.0)</code> is 1.0 <code>log10 (100.0)</code> is 2.0
<code>fabs (x)</code>	absolute value of x as a floating-point number	<code>fabs (13.5)</code> is 13.5 <code>fabs (0.0)</code> is 0.0

math.h

Function	Description	Example
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0
<code>pow(x, y)</code>	x raised to power ____	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of _____ as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Outline

- Strutture e funzioni
- Modulare: le funzioni di sistema
- Definire nuove funzioni
- Le funzioni e lo stack

Programmazione modulare

- Finora abbiamo scritto tutto il nostro codice nella funzione *main()*
- Alcuni algoritmi come lo scambio di variabili sono richiesti più volte nel programma
- Abbiamo copiato il blocco di codice dello scambio variabili dove richiesto
- Codice identico in tre punti del programma, cambia solo nome delle variabili

```
...  
if (z < x) {  
    temp = z;  
    z = x;  
    x = temp;  
}  
if (z < y) {  
    temp = z;  
    z = y;  
    y = temp;  
}  
if (y < x) {  
    temp = y;  
    y = x;  
    x = temp;  
}  
...
```

Programmazione modulare

- Copiare lo stesso codice è un cattivo approccio per diverse ragioni:
 - Compattezza del codice
 - Manutenibilità del software
 - Riutilizzabilità del codice
 - Modularità dello sviluppo

```
...  
if (z < x) {  
    temp = z;  
    z = x;  
    x = temp;  
}  
if (z < y) {  
    temp = z;  
    z = y;  
    y = temp;  
}  
if (y < x) {  
    temp = y;  
    y = x;  
    x = temp;  
}  
...
```

Le funzioni

- Finora tutto il nostro codice si trovava nella funzione *main()*
- Le funzioni rappresentano un blocco di codice invocabile dall'esterno con *parametri di input* e *valore di ritorno*

`y = square(x);`

y è il valore ritornato *x* è il parametro

- Possono accettare in input più **parametri**

`printf("%d diviso %d: %d\n", x, y, x/y);`

- Possono non restituire alcun valore

`scanf("%d", &x);`

Le funzioni

- Il *prototipo* della funzione viene **dichiarato** prima del `main()`
 - Permette al compilatore controlli correttezza numero e tipo parametri input e su tipo di valore ritornato
- La funzione viene **definita** dopo la funzione *main()*
 - La definizione di funzione deve essere concorde con il prototipo per numero e per tipo di parametri di input e tipo di parametro di ritorno
- Valgono le regole viste per la **nomomenclatura** degli identificatori
 - nome inizia per minuscolo, niente spazi, camelToe convention
 - una sola funzione con un dato nome

square

```
1. // fig05_01.c
2. // Creating and using a function.
3. #include <stdio.h>
4.
5. int square(int number); // function prototype
6.
7. int main(void) {
8.     // loop 10 times and calculate and output square of x each time
9.     for (int x = 1; x <= 10; ++x) {
10.         printf("%d  ", square(x)); // function call
11.     }
12.
13.     puts("");
14. }
15.
16. // square function definition returns the square of its parameter
17. int square(int number) { // number is a copy of the function's argument
18.     return number * number; // returns square of number as an int
19. }
```

square

```
1. // fig05_01.c
2. // Creating and using a function.
3. #include <stdio.h>
4.
5. int square(int number); // function prototype
6.
7. int main(void) {
8.     // loop 10 times and calculate and output square of x each time
9.     for (int x = 1; x <= 10; ++x) {
10.        printf("%d ", square(x)); // function call
11.    }
12.
13.    puts("");
14.}
15.
16. // square function definition returns the square of its parameter
17. int square(int number) { // number is a copy of the function's argument
18.     return number * number; // returns square of number as an int
19.}
```

Dichiarazione
del prototipo

Chiamata della
funzione

Definizione della
funzione

square

chiamata

- square is **invoked** or **called** in line 10:
 printf("%d ", square(x)); // function call
- Receives a copy of the argument x's value in the parameter number (line 17).
- Calculates number * number and passes the result back to line 10 in main where square was invoked.
- Line 10 passes square's result to function printf, which displays the result on the screen

```
1. // fig05_01.c
2. // Creating and using a function.
3. #include <stdio.h>
4.
5. int square(int number); // function prototype
6.
7. int main(void) {
8.     // loop 10 times and calculate and output square of x each time
9.     for (int x = 1; x <= 10; ++x) {
10.        printf("%d  ", square(x)); // function call
11.    }
12.
13.    puts("");
14.}
15.
16.// square function definition returns the square of its parameter
17.int square(int number) { // number is a copy of the function's argument
18.    return number * number; // returns square of number as an int
19.}
```